

AIDOS Technical Dossier

Agentic AI Distribution Intelligence Platform for FMCG — full technical documentation: architecture, algorithms, real authentication, external services, and the AI-assisted build process (with the actual prompts used at each stage).

PREPARED BY

Vurimi

REPOSITORY

github.com/vurimibhuvana2016/aidos-command-center

LIVE APP

aidos-command-center.vercel.app

STATUS

MVP • real accounts, local data

CONTENTS

1. Build timeline
2. MVP scope & wedge
3. System architecture
4. Technology stack
5. Core algorithm
6. Feature walkthrough
7. Authentication flow
8. Codebase structure
9. External services configured
10. AI-assisted build log & prompts
11. Testing & verification
12. Known limitations
13. Roadmap

Build timeline

What shipped, in order — from a static prototype to a deployed product with real accounts.

1. Deployment

Existing static React/Vite prototype pushed to GitHub, connected to Vercel for free hosting with auto-deploy on every push.

2. Feature audit & test manifest

Full walkthrough of every screen against the actual risk-engine output, to verify the MVP behaved as documented.

3. Demo login gate + flexible Data Hub fields

Cosmetic name/workspace entry screen; CSV import extended to auto-capture columns beyond the core schema, plus a UI to add custom fields by hand.

4. Excel import

Data Hub accepts .xlsx/.xls alongside CSV, via a lazy-loaded SheetJS parser so it doesn't bloat the initial page load.

5. Client-side Google Sign-In

A first pass at "Continue with Google" — decoded a Google ID token in the browser for display purposes only, explicitly not server-verified.

6. Functional notification bell

Replaced a decorative, non-functional bell icon with a real dropdown of recent activity.

7. Per-workspace demo data generator

Every trial workspace now gets its own distinct (but reproducible) set of sample SKUs, seeded from the workspace name, instead of one shared static dataset.

8. Real authentication (Supabase)

Replaced the cosmetic login entirely: real email/password registration with email confirmation, real sign-in, and a properly server-verified "Continue with Google" — while deliberately leaving inventory data on local storage for now.

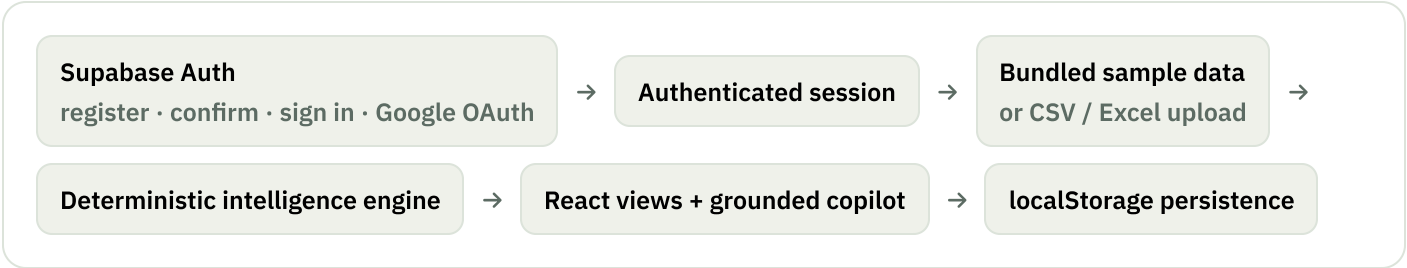
MVP scope & wedge

Per the lab's own scoping method — pick one entry-point workflow rather than the full canvas (forecasting + inventory + routing + computer vision) — this MVP narrows to a single wedge: **demand-and-risk visibility with an explainable copilot, for one distribution depot**, now gated behind real user accounts. The user journey proven end to end: a signal is detected → AIDOS explains why in plain language → recommends one concrete action → a named human approves it → the approval is recorded in an auditable trail.

Still deliberately out of scope: route optimization, computer vision, multi-region rollout, live ERP integration, and — for now — moving inventory data itself into a shared database (see §13 Roadmap).

System architecture

Authentication is now real and server-backed (Supabase). Application data is deliberately still local-only — a scoped decision, not an oversight, to keep this stage of the build small and shippable.



Every screen still reads from the same computed `Insight[]` array — one source of truth for risk, unrelated to who's logged in. Authentication answers "who is this," not "what should they see" — everyone signed in currently sees their own browser's local demo dataset, not a shared company dataset.

Technology stack

LAYER	CHOICE	VERSION	WHY
UI framework	React	19.1.0	Component model matches the screen-based navigation.
Language	TypeScript	5.8.3	Static types on the data model catch schema mistakes at build time.
Build tool	Vite	6.4.3	Sub-second dev reloads; static dist/ output for free hosting.
Authentication	Supabase Auth	js SDK 2.116.0	Real email/password registration with confirmation emails, plus server-verified Google OAuth — free tier, no custom backend to run.
Charts	Recharts	3.10.1	Demand trend, forecast, and risk-radar visualizations.
Icons	lucide-react	0.468.0	Consistent icon set across all screens.
Spreadsheet import	xlsx (SheetJS)	CDN build	Parses .xlsx/.xls uploads client-side, lazy-loaded on first use. Installed from SheetJS's own CDN — the npm-registry build carries an unpatched high-severity advisory.
Testing	Vitest + jsdom	3.2.7	Unit tests for the risk/reorder engine, isolated from the UI.
Hosting	Vercel	—	Free static hosting with Git-triggered auto-deploy.

Core algorithm

Unchanged by the authentication work — all operational intelligence still lives in one pure function, `analyzeProduct()` in `src/lib/intelligence.ts`, deliberately independent of who's logged in or how. No machine learning — three explainable calculations, checked in priority order:

METRIC	FORMULA	MEANING
Days cover	<code>stock ÷ daily sales</code>	How long current stock lasts at observed velocity.
Reorder quantity	<code>ceil(demand over lead time + safety stock) - stock</code>	Units needed today to stay above the safety-stock line through the next delivery.
Risk cascade	<code>expiry exposure → stock-out risk → excess inventory → freshness watch → healthy</code>	First matching rule wins; every product gets exactly one risk label and one plain-language reason.

The copilot (`answerQuestion()`, same file) never generates its own numbers — it selects a product from this pre-computed array by keyword match and narrates fields that already exist.

Feature walkthrough

01 Login / Register
Real email/password registration with confirmation email, sign-in, and server-verified Google OAuth via Supabase.

02 Command center
KPI tiles, demand trend, risk-radar donut, top signals ranked by financial impact.

03 Inventory
Searchable, filterable SKU table with a drill-down evidence panel, including editable custom fields.

04 Forecast
7-day per-SKU demand outlook with named drivers and a reorder recommendation.

05 Action center
Every non-healthy SKU as an approval card feeding a real decision trail.

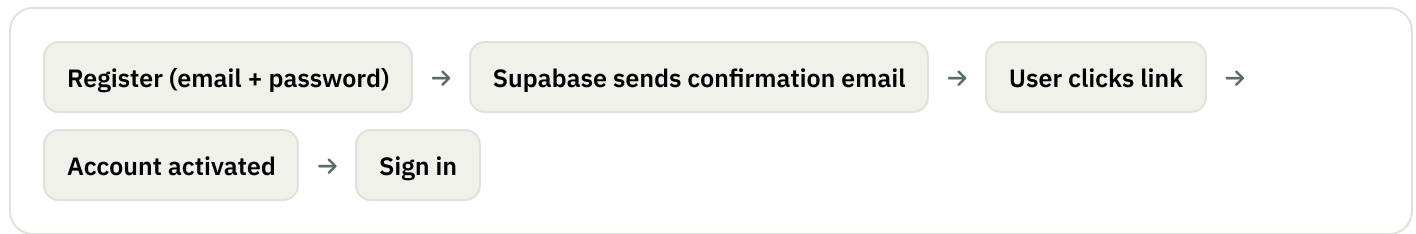
06 Data hub
CSV and Excel import with schema validation, plus custom fields — by hand or auto-detected from extra file columns.

07 Ask AIDOS
Grounded copilot answering only from on-screen evidence, with a visible reasoning trail.

08 Notifications
A working dropdown of real recent activity, replacing a purely decorative bell icon.

Authentication flow

Built with Supabase Auth's email provider (confirmation required) and Google OAuth provider, reusing one Google Cloud OAuth Client for both this and the earlier client-side experiment.



src/App.tsx — session lifecycle

```

useEffect(() => {
  supabase.auth.getSession().then(({ data }) => {
    setAuthUser(data.session?.user ?? null);
    setAuthLoading(false);
  });
  const { data: sub } = supabase.auth.onAuthStateChange((_event, session) => {
    setAuthUser(session?.user ?? null);
  });
  return () => sub.subscription.unsubscribe();
}, []);

```

One listener handles every transition — registration, sign-in, sign-out, and the redirect back from Google or an email confirmation link — so the rest of the app just reacts to "is there a user," never manages auth state itself. Name/workspace are stored as Supabase user metadata (options.data at sign-up) rather than a database table, since no app data is server-side yet.

Codebase structure

```
src/
├─ App.tsx – routes between screens, owns top-level state, auth lifecycle
├─ main.tsx – React entry point
├─ styles.css – design tokens + component styles
├─ types.ts – Product, Insight, Activity, Page, DisplaySession
├─ components/
│   ├─ Login.tsx – register / sign-in / check-email / Google, via Supabase
│   ├─ Sidebar.tsx – navigation + workspace/user identity + logout
│   ├─ Copilot.tsx – Ask AIDOS chat panel
│   └─ MetricCard.tsx, RiskPill.tsx – shared UI atoms
├─ data/demo.ts – static fallback dataset (8 SKUs)
├─ lib/intelligence.ts – risk/reorder engine + copilot logic
├─ lib/demoGenerator.ts – seeded per-workspace dummy data
├─ lib/supabase.ts – Supabase client (URL + anon key from env)
└─ tests/intelligence.test.ts – unit tests for the engine
```

External services configured

SERVICE	PURPOSE	PLAN
GitHub	Source control, triggers Vercel deploys on push	Free
Vercel	Static hosting, auto-deploy, environment variables	Hobby (free)
Google Cloud	OAuth 2.0 Client (Web application) for Google sign-in — Client ID is public, Client Secret lives only in Supabase's dashboard	Free
Supabase	Auth (email confirmation + Google OAuth provider)	Free tier

Config values needed to run this project live in environment variables (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY), documented in .env.example and never committed as real values (.env is gitignored).

AI-assisted build log & prompts

In the spirit of AIDOS_DVL_Playbook.md's "build with AI tools only" track — this is the actual sequence of prompts given to Claude Code that drove this build, paraphrased for clarity, with what each one produced.

STAGE 1 — GO LIVE

"I want to deploy this application and make it available for multiple users — help me do this step by step."

Result: Node.js environment set up, project pushed to a new GitHub repo, imported into Vercel with auto-deploy — first live public link.

STAGE 2 — UNDERSTAND WHAT WAS BUILT

"Explain the functionality and help me test every feature — how it benefits, where to use it."

Result: A published test-checklist artifact with an answer key computed from the actual risk engine, so every screen could be verified against real expected output.

STAGE 3 — FLEXIBILITY + A LOGIN GATE

"Add flexibility for adding fields in Data Hub. Add authentication — login/logout."

Result: Clarified scope first (demo-style vs. real auth; CSV-only vs. hand-added fields) via plan mode, then shipped a cosmetic login screen and a custom-fields system for Data Hub.

STAGE 4 — BROADEN FILE SUPPORT

"Like data sets or CSV file, Excel, and flexibility in adding columns."

Result: Excel (.xlsx/.xls) import added alongside CSV, reusing the same validation path; the parser library was lazy-loaded after a bundle-size warning surfaced a real regression.

STAGE 5 — PERSONALIZE THE DEMO

"Give me some dummy data for demo trials — user specific, they get it."

Result: A seeded data generator so each workspace name deterministically produces its own distinct sample dataset instead of one shared static set.

STAGE 6 — REAL ACCOUNTS

"One should be able to register. After registration, an email should go to the registered email ID. After confirmation, it should be activated."

Result: The cosmetic login was replaced entirely with Supabase Auth — real registration, confirmation email, activation, and a properly server-verified Google sign-in, scoped deliberately to leave inventory data local for now.

STAGE 7 — DOCUMENT IT

"Give the entire tech document, coding step by step, everything to compile, including prompts."

Result: This document.

11

Testing & verification

CHECK	TOOL	RESULT
Unit tests — risk/reorder logic	<code>vitest run</code>	4 / 4 passing
Type safety	<code>tsc -b</code>	no errors
Production build	<code>vite build</code>	builds clean
Dependency audit	<code>npm audit</code>	0 high, 2 moderate (dev-only, test runner)
Manual — register → confirm → sign in	Real inbox + Supabase dashboard	verified
Manual — Google OAuth round trip	Live redirect through Google	verified

Known limitations at this stage

Deliberately scoped, not overclaimed

- **Data locality** — inventory/product/activity data still lives in browser `localStorage`, per device. Accounts are real; the data behind them isn't shared or synced yet.
- **No workspace isolation** — every signed-in user currently generates their own local demo dataset; there's no server-side concept of "this company's data" yet.
- **Copilot** — keyword matching over pre-computed data, not an LLM call; guarantees no invented figures, at the cost of only handling a fixed set of question shapes.
- **No password reset flow yet** — registration, confirmation, and sign-in are complete; "forgot password" wasn't in scope for this pass.

Roadmap

Shared data

Move inventory data from `localStorage` into Supabase's Postgres database, with Row Level Security keyed on a real `workspace_id` so each company's data is private and synced across devices.

Real copilot

Move the LLM call server-side (Supabase Edge Function), retrieving deterministic metrics first and having the model explain them — never calculate them.

Password reset

Standard "forgot password" email flow, same Supabase Auth foundation.

Governance

Role checks, audit logs, rate limits, backups, and an explicit data-retention policy before any real pilot.

Live application and source are linked at the top of this document. This dossier reflects the codebase through the authentication milestone described in `AIDOS_DVL_Playbook.md`, Stage 3–4.